

UNIT-I: Overview and Introduction to Software Engineering

1. Introduction to Professional Software Development

Software engineering is a discipline concerned with the theories, methods, and tools essential for professional software development. It encompasses all aspects of software production, from the initial stages of system specification through to maintaining the system after it has been deployed.

1.1. Importance of Software in Modern Economies The economies of all developed nations are significantly dependent on software. An increasing number of systems are controlled by software. Expenditure on software constitutes a substantial fraction of the Gross National Product (GNP) in all developed countries. Society and individuals increasingly rely on advanced software systems, necessitating the ability to produce reliable, trustworthy systems economically and quickly.

1.2. Software Costs Software costs frequently dominate the overall costs of computer systems. For instance, the cost of software on a Personal Computer (PC) is often higher than the hardware cost. Furthermore, maintaining software typically costs more than its initial development. For systems with a long operational life, maintenance expenses can be several times greater than development costs. Software engineering focuses on cost-effective software development. Roughly 60% of software costs are attributable to development, while 40% are for testing. For customized software, evolution costs often surpass development costs. In the long run, employing software engineering methods and techniques for software systems is generally more economical than treating software development as a personal programming project. For most system types, the majority of costs arise from changing the software after it has been put into use.

1.3. Software Products Software products can be broadly categorized into two types:

- **Generic Products:**
 - These are stand-alone systems that are marketed and sold to any customer desiring to purchase them.
 - **Examples:** PC software such as graphics programs, project management tools, CAD software, and software for specific markets like appointment systems for dentists.
 - **Product Specification:** The ownership of the software's functional specification resides with the software developer. Decisions regarding software changes are made by the developer.
- **Customized Products:**

- This refers to software commissioned by a specific customer to fulfill their unique requirements.
- **Examples:** Embedded control systems, air traffic control software, and traffic monitoring systems.
- **Product Specification:** The ownership of the software's functional specification rests with the customer for whom the software is developed. The customer makes decisions on required software changes.

1.4. Fundamental Concepts of Software Engineering Several frequently asked questions help define the core aspects of software engineering:

- **What is software?** Software comprises computer programs and their associated documentation. Software products can be developed either for a specific customer or for a general market.
- **What are the attributes of good software?** Good software should deliver the required functionality and performance to the user. Additionally, it must be maintainable, dependable, and usable.
- **What is software engineering?** Software engineering is an engineering discipline concerned with all aspects of software production. It involves using appropriate theories and methods to solve problems, while considering organizational and financial constraints. This includes not only the technical process of development but also project management and the creation of tools and methods to support software production.
- **What are the fundamental software engineering activities?** The core activities are software specification, software development, software validation, and software evolution. These high-level activities are integral to all software processes.
- **What is the difference between software engineering and computer science?** Computer science primarily focuses on theoretical aspects and fundamentals. In contrast, software engineering is concerned with the practicalities of developing and delivering useful software.
- **What is the difference between software engineering and system engineering?** System engineering addresses all aspects of computer-based systems development, including hardware, software, and process engineering. Software engineering is a specific part of this broader process.
- **What are the key challenges facing software engineering?** The primary challenges include coping with increasing diversity, meeting demands for reduced delivery times, and developing trustworthy software.
- **What are the best software engineering techniques and methods?** There isn't one universal best method. While all software projects require

professional management and development, different techniques are suitable for different types of systems. For instance, games are typically developed using a series of prototypes, whereas safety-critical control systems necessitate a complete and analyzable specification. Therefore, it is not possible to state that one method is inherently superior to another.

- **What differences has the web made to software engineering?** The web has enabled the availability of software services and the potential for developing highly distributed service-based systems. Web-based systems development has led to significant advancements in programming languages and software reuse. The web now serves as a platform for running applications, with organizations increasingly developing web-based systems instead of local ones.

1.5. Essential Attributes of Good Software Good software possesses several critical characteristics:

- **Maintainability:** Software should be constructed in a way that allows it to evolve to meet the changing needs of customers. This attribute is crucial because software change is an unavoidable requirement in a dynamic business environment.
- **Dependability and Security:** Software dependability encompasses characteristics such as reliability, security, and safety. Dependable software should prevent physical or economic damage in the event of system failure. Furthermore, malicious users should not be able to access or damage the system.
- **Efficiency:** Software should utilize system resources, such as memory and processor cycles, without wasteful consumption. Efficiency includes responsiveness, processing time, and memory utilization, among others.
- **Acceptability:** Software must be acceptable to its intended users. This implies that it must be understandable, usable, and compatible with other systems the users employ.

1.6. Software Process Activities The software process involves a series of interconnected activities:

- **Software Specification:** This is where customers and engineers collaboratively define the software to be produced and the constraints governing its operation.
- **Software Development:** This phase involves the design and programming of the software.
- **Software Validation:** In this activity, the software is checked to ensure it meets the customer's requirements.

- **Software Evolution:** This refers to the modification of software to accommodate changing customer and market requirements.

1.7. General Issues Affecting Most Software Most software development is influenced by several pervasive issues:

- **Heterogeneity:** Increasingly, systems are required to operate as distributed systems across networks that incorporate diverse types of computers and mobile devices.
- **Business and Social Change:** Both business and society are undergoing rapid transformation due to emerging economies and new technologies. This necessitates the ability to quickly modify existing software and develop new software.
- **Security and Trust:** Given that software is deeply intertwined with almost every aspect of our lives, it is essential that we can trust the software we use.

1.8. Software Engineering Diversity and Application Types There are numerous types of software systems, and consequently, no single universal set of software techniques applies to all of them. The choice of software engineering methods and tools depends on the specific application being developed, the customer's requirements, and the development team's background.

Common application types include:

- **Stand-alone applications:** These are application systems that execute on a local computer, such as a PC, encompassing all necessary functionality without requiring network connection.
- **Interactive transaction-based applications:** These applications run on a remote computer and are accessed by users from their own PCs or terminals, including web applications like e-commerce systems.
- **Embedded control systems:** These are software control systems designed to manage and control hardware devices. Numerically, embedded systems are likely the most prevalent type of system.
- **Batch processing systems:** These business systems are designed to process data in large batches, handling numerous individual inputs to generate corresponding outputs.
- **Entertainment systems:** Primarily for personal use, these systems are intended to entertain the user.
- **Systems for modeling and simulation:** Developed by scientists and engineers, these systems model physical processes or situations that involve many separate, interacting objects.

- **Data collection systems:** These systems gather data from their environment using sensors and transmit that data to other systems for processing.
- **Systems of systems:** These are complex systems composed of a number of other software systems.

1.9. Software Engineering Fundamentals Regardless of the development techniques employed, some fundamental principles apply to all types of software systems:

- Systems should be developed using a managed and well-understood development process, though different processes are used for different software types.
- Dependability and performance are crucial for all system types.
- Understanding and managing the software specification and requirements (i.e., what the software should do) are important.
- Where appropriate, developers should reuse existing software rather than creating new software from scratch.

1.10. Software Engineering and the Web The web has become a dominant platform for running applications, leading organizations to increasingly develop web-based systems over local systems.

- **Web Services:** These allow application functionality to be accessed over the web.
- **Cloud Computing:** This approach involves the provision of computer services where applications run remotely on the "cloud". Users typically pay based on usage rather than buying software outright.

1.11. Web Software Engineering Specific characteristics of web software engineering include:

- **Software Reuse:** This is the primary approach for constructing web-based systems, emphasizing assembling systems from pre-existing software components and systems.
- **Incremental Development:** Web-based systems should be developed and delivered incrementally. It is widely acknowledged that specifying all requirements for such systems in advance is impractical.
- **User Interface Constraints:** User interfaces are limited by the capabilities of web browsers. While technologies like AJAX enable the creation of rich interfaces within a web browser, they remain challenging to implement. Web forms with local scripting are more commonly used.

Despite these specific considerations, the fundamental principles of software engineering, as discussed, are equally applicable to web-based systems as they are to other types of software systems.

2. Software Engineering Ethics

Software engineering involves broader responsibilities than merely applying technical skills. Software engineers are expected to behave honestly and ethically to gain respect as professionals. Ethical behavior extends beyond legal compliance; it entails adhering to a set of morally correct principles.

2.1. Issues of Professional Responsibility Several key areas define professional responsibility for software engineers:

- **Confidentiality:** Engineers are generally obligated to respect the confidentiality of their employers or clients, regardless of whether a formal confidentiality agreement has been signed.
- **Competence:** Engineers should accurately represent their level of competence and should not knowingly accept work that falls outside their area of expertise.
- **Intellectual Property Rights (IPR):** Engineers must be knowledgeable about local laws concerning the use of intellectual property, such as patents and copyright. They should diligently ensure that the intellectual property of their employers and clients is protected.
- **Computer Misuse:** Software engineers should not use their technical skills to misuse other people's computers. Computer misuse can range from relatively trivial acts (e.g., playing games on an employer's machine) to extremely serious offenses (e.g., disseminating viruses).

2.2. ACM/IEEE Code of Ethics The professional societies in the United States have collaborated to develop a code of ethical practice for software engineers. Members of these organizations commit to this code of practice upon joining.

2.2.1. Rationale for the Code of Ethics Computers play a central and increasingly significant role in commerce, industry, government, medicine, education, entertainment, and society at large. Software engineers are individuals who contribute to the analysis, specification, design, development, certification, maintenance, and testing of software systems, either through direct participation or by teaching. Due to their involvement in developing software systems, software engineers possess considerable opportunities to do good or cause harm, enable others to do good or cause harm, or influence others to do good or cause harm. To maximize the beneficial use of their efforts, software

engineers must commit to establishing software engineering as a beneficial and respected profession.

2.2.2. Preamble to the Code The short version of the Code of Ethics and Professional Practice summarizes aspirations at a high level of abstraction. The clauses in the full version provide examples and details on how these aspirations translate into action for software engineering professionals. The aspirations without details can become empty, and details without aspirations can become legalistic. Together, aspirations and details form a cohesive code.

Software engineers are mandated to commit themselves to ensuring that the analysis, specification, design, development, testing, and maintenance of software contribute to a beneficial and respected profession. In alignment with their commitment to the health, safety, and welfare of the public, software engineers must adhere to the following Eight Principles.

2.2.3. The Eight Ethical Principles The Code contains eight principles guiding the behavior and decisions of professional software engineers, including practitioners, educators, managers, supervisors, policy makers, trainees, and students.

1. **PUBLIC:** Software engineers shall consistently act in the public interest.
2. **CLIENT AND EMPLOYER:** Software engineers shall act in a manner that serves the best interests of their client and employer, provided it is consistent with the public interest.
3. **PRODUCT:** Software engineers shall ensure that their products and any related modifications meet the highest possible professional standards.
4. **JUDGMENT:** Software engineers shall uphold integrity and independence in their professional judgment.
5. **MANAGEMENT:** Software engineering managers and leaders shall endorse and promote an ethical approach to the management of software development and maintenance.
6. **PROFESSION:** Software engineers shall advance the integrity and reputation of the profession in a manner consistent with the public interest.
7. **COLLEAGUES:** Software engineers shall be fair to and supportive of their colleagues.
8. **SELF:** Software engineers shall engage in lifelong learning regarding their professional practice and shall promote an ethical approach to the profession.

2.3. Ethical Dilemmas Software engineers may face various ethical dilemmas, such as:

- Disagreement in principle with the policies of senior management.
 - An employer acting unethically, for example, by releasing a safety-critical system without completing its testing.
 - Participation in the development of military weapons systems or nuclear systems.
-

3. Case Studies

Three illustrative case studies are used throughout the book to demonstrate software engineering concepts:

- A personal insulin pump control system.
- A mental health care patient management system.
- A wilderness weather station.

3.1. Case Study 1: A Personal Insulin Pump Control System This is an embedded system used by diabetics to maintain blood glucose control.

3.1.1. System Functionality The system collects data from a blood sugar sensor and calculates the amount of insulin required for injection. This calculation is based on the rate of change of blood sugar levels. It then sends signals to a micro-pump to deliver the correct dose of insulin.

3.1.2. Safety-Critical Nature This system is safety-critical. Low blood sugar levels can lead to brain malfunctioning, coma, and even death. Conversely, high blood sugar levels have long-term consequences such as eye and kidney damage.

3.1.3. Essential High-Level Requirements

- The system must be available to deliver insulin whenever required.
- The system must perform reliably and deliver the correct amount of insulin to counteract the current blood sugar level.
- Therefore, the system must be designed and implemented to consistently meet these requirements.

3.1.4. Hardware Architecture (Visual Representation in Source) *(The source includes a diagram "Insulin pump hardware architecture" on page 35, which visually depicts the components of the system).*

3.1.5. Activity Model (Visual Representation in Source) *(The source includes a diagram "Activity model of the insulin pump" on page 35, showing the flow of activities within the system).*

3.2. Case Study 2: A Patient Information System for Mental Health Care (MHC-PMS) This is a medical information system designed to support mental health care by maintaining information about patients suffering from mental health problems and their received treatments.

3.2.1. Operational Context Most mental health patients do not require dedicated hospital treatment but need to attend specialist clinics regularly to meet a doctor familiar with their problems. To enhance accessibility, these clinics are operated not only in hospitals but also in local medical practices or community centers.

3.2.2. System Design and Connectivity The MHC-PMS is an information system intended for use in these clinics. It leverages a centralized database for patient information but is also designed to run on a PC, allowing access and use from sites without secure network connectivity. When local systems have secure network access, they use patient information from the database. However, they can download and utilize local copies of patient records when disconnected.

3.2.3. MHC-PMS Goals

- To generate management information enabling health service managers to assess performance against local and government targets.
- To provide medical staff with timely information to support patient treatment.

3.2.4. Key Features of MHC-PMS

- **Individual Care Management:** Clinicians can create patient records, edit system information, view patient history, and access data summaries to quickly understand key problems and prescribed treatments.
- **Patient Monitoring:** The system monitors patient records involved in treatment and issues warnings if potential problems are detected.
- **Administrative Reporting:** The system generates monthly management reports detailing information such as the number of patients treated at each clinic, patients entering and leaving care, patients sectioned, prescribed drugs, and their costs.

3.2.5. MHC-PMS Concerns

- **Privacy:** Patient information must remain confidential and never be disclosed to unauthorized individuals, only to authorized medical staff and the patient themselves.
- **Safety:** Some mental illnesses can lead patients to become suicidal or pose a danger to others. Wherever possible, the system should warn medical staff about potentially suicidal or dangerous patients. The system's availability is also crucial for safety, as its unavailability could compromise safety and prevent the correct medication from being prescribed.

3.2.6. Organization of the MHC-PMS (Visual Representation in Source)

(The source includes a diagram "The organization of the MHC-PMS" on page 38, showing its structural components).

3.3. Case Study 3: A Wilderness Weather Station This case study involves a country deploying several hundred weather stations in remote wilderness areas.

3.3.1. System Functionality Weather stations collect data from a set of instruments that measure temperature, pressure, sunshine, rainfall, wind speed, and wind direction. Each instrument is controlled by a software system that periodically takes parameter readings and manages the collected data.

3.3.2. The Weather Station's Environment (Visual Representation in Source) *(The source includes a diagram "The weather station's environment" on page 42, illustrating the context of the station).*

3.3.3. Components of the Weather Information System The overall weather information system comprises three main parts:

- **The Weather Station System:** This component is responsible for collecting weather data, performing initial data processing, and transmitting it to the data management system.
- **The Data Management and Archiving System:** This system collects data from all wilderness weather stations, conducts data processing and analysis, and archives the data.
- **The Station Maintenance System:** This system communicates with all wilderness weather stations via satellite to monitor their health and report problems.

3.3.4. Additional Software Functionality The software within the weather station system also provides additional functionalities:

- Monitoring the instruments, power, and communication hardware, and reporting faults to the management system.

- Managing the system's power, ensuring batteries are charged when environmental conditions permit, and generators are shut down in potentially damaging weather conditions like high wind.
- Supporting dynamic reconfiguration, which involves replacing parts of the software with new versions and switching in backup instruments if system failure occurs.

UNIT-I: Overview and Software Processes

1. Introduction to Professional Software Development

This unit provides an overview of professional software development, software engineering ethics, and related case studies. It delves into software processes, including models, activities, coping with change, and process improvement.

2. Software Processes

2.1. Definition of a Software Process A software process is defined as a structured set of activities that are required to develop a software system. While many different software processes exist, they all typically involve four fundamental activities:

- **Specification:** This activity involves defining precisely what the system is intended to do.
- **Design and Implementation:** This phase focuses on defining the overall organization or architecture of the system and subsequently building or coding the system.
- **Validation:** This crucial activity involves checking and confirming that the developed software system fulfills the customer's requirements and expectations.
- **Evolution:** This ongoing activity entails changing or adapting the system in response to evolving customer needs and requirements.

2.2. Software Process Models A software process model is an abstract representation of a process. It offers a description of a process from a particular perspective or viewpoint.

2.3. Software Process Descriptions When discussing and describing software processes, the focus is usually on the specific activities involved, such as specifying a data model or designing a user interface, and the sequential ordering of these activities. Process descriptions may also encompass:

- **Products:** These are the tangible outcomes or artifacts resulting from a specific process activity.

- **Roles:** These define the responsibilities assigned to the individuals involved in the software development process.
- **Pre- and Post-conditions:** These are statements that must be true before a process activity can begin (pre-conditions) and statements that are true after a process activity has been completed or a product has been generated (post-conditions).

2.4. Plan-driven vs. Agile Processes Software processes can broadly be categorized into two main types based on their approach to planning and change:

- **Plan-driven Processes:** In these processes, all activities are meticulously planned in advance, and the progress of the project is systematically measured against this pre-established plan.
- **Agile Processes:** These processes feature incremental planning, making it significantly easier to adapt and change the process in response to evolving customer requirements.
- **Practical Application:** In real-world software development, most practical processes integrate elements from both plan-driven and agile approaches. There is no single "right" or "wrong" software process; the most suitable approach depends on the project's context.

2.5. Common Software Process Models Several abstract software process models guide the development lifecycle:

- **The Waterfall Model:** This is a plan-driven model characterized by distinct and separate phases for specification and development.
- **Incremental Development:** In this model, specification, development, and validation activities are interleaved. It can be implemented in either a plan-driven or an agile manner.
- **Reuse-oriented Software Engineering:** This approach involves assembling the software system primarily from existing components. It can also be either plan-driven or agile.
- **Hybrid Approaches:** In practice, the development of most large software systems utilizes a process that combines elements from all of these models.

3. The Waterfall Model

3.1. Waterfall Model Phases The waterfall model is characterized by a series of distinct, sequential phases:

- **Requirements Analysis and Definition:** This initial phase involves gathering, analyzing, and formally documenting the system's requirements.
- **System and Software Design:** In this phase, the overall system architecture and detailed software designs are created based on the defined requirements.
- **Implementation and Unit Testing:** This phase involves coding the software components and testing them individually (unit testing) to ensure they function correctly.
- **Integration and System Testing:** Once individual units are tested, they are integrated, and the entire system is tested as a whole to verify that all components work together as intended.
- **Operation and Maintenance:** After deployment, this ongoing phase involves the system's operational use, along with maintenance activities such as bug fixes, enhancements, and adaptations to new environments.

3.2. Waterfall Model Problems The primary drawback of the waterfall model is the inherent difficulty in accommodating changes once the process is underway. A fundamental principle is that one phase must be completed before the next can begin, which makes responding to changing requirements challenging.

- **Inflexibility:** Its rigid partitioning into distinct stages makes it difficult to adapt to evolving customer requirements.
- **Requirement Stability:** This model is only appropriate when requirements are exceptionally well-understood at the outset and when changes during the design process are expected to be very limited.
- **Limited Applicability:** Few business systems inherently have stable requirements, making the waterfall model less suitable for most business applications.
- **Specific Use Case:** The waterfall model is primarily used for large-scale systems engineering projects, especially when a system is developed across multiple geographically dispersed sites. In such scenarios, its plan-driven nature aids in coordinating the work effectively.

4. Incremental Development

4.1. Benefits of Incremental Development Incremental development offers several advantages:

- **Reduced Cost of Change:** The cost associated with accommodating changing customer requirements is significantly reduced because less analysis and documentation need to be redone compared to the waterfall model.

- **Enhanced Customer Feedback:** It facilitates easier and more frequent customer feedback on the development work. Customers can provide comments on software demonstrations and observe the progress of implementation.
- **Rapid Delivery and Deployment:** This model allows for quicker delivery and deployment of useful software to the customer. Customers can start using and gaining value from the software much earlier than with a waterfall process.

4.2. Problems with Incremental Development Despite its benefits, incremental development also presents challenges:

- **Lack of Process Visibility:** The process can be less visible to managers. Managers typically require regular deliverables to measure progress, but producing documentation for every quick iteration of a system may not be cost-effective.
- **System Structure Degradation:** As new increments are added, the system's overall structure tends to degrade. Without dedicated time and resources allocated for refactoring to improve the software's internal quality, continuous change can corrupt its structure, making future software changes increasingly difficult and costly.

5. Reuse-Oriented Software Engineering

5.1. Core Concept Reuse-oriented software engineering is an approach based on systematic reuse, where software systems are constructed by integrating existing components or Commercial-Off-The-Shelf (COTS) systems.

5.2. Process Stages The process typically involves four key stages:

- **Component Analysis:** Identifying and evaluating available components for suitability.
- **Requirements Modification:** Adapting the system requirements to align with the capabilities and constraints of the chosen reusable components.
- **System Design with Reuse:** Designing the system architecture by incorporating and integrating the selected components.
- **Development and Integration:** Developing any necessary new components and integrating all components (existing and new) to form the complete system.

5.3. Types of Software Components for Reuse Various types of software components can be reused:

- **Web Services:** These are services developed according to service standards, designed for remote invocation over the web.
- **Object Collections:** These are packages of objects developed to be integrated within a component framework, such as .NET or J2EE.
- **Stand-alone Software Systems (COTS):** These are complete, pre-built software systems that can be configured for use in a specific environment.

6. Software Design and Implementation

6.1. Definition Software design and implementation constitute the process of transforming a system specification into an executable software system.

- **Software Design:** This involves creating a software structure that effectively realizes or fulfills the defined system specification.
- **Implementation:** This stage translates the designed software structure into an executable program or code. The activities of design and implementation are intimately related and are often interleaved in practice.

6.2. A General Model of the Design Process The design process typically involves several interconnected activities:

- **Architectural Design:** This involves identifying the overarching structure of the system, determining its principal components (also known as sub-systems or modules), defining their relationships, and planning how these components will be distributed.
- **Interface Design:** This activity focuses on precisely defining the interfaces that enable communication and interaction between the various system components.
- **Component Design:** For each individual system component, this activity details how it will operate and perform its specific functions.
- **Database Design:** This involves designing the system's data structures and determining how these structures will be represented and managed within a database.

7. Software Validation

7.1. Verification and Validation (V&V) Verification and Validation (V&V) is a critical set of activities intended to demonstrate that a system conforms to its specified requirements and successfully meets the needs of the system customer. It encompasses various checking and review processes, as well as systematic system testing.

- **System Testing:** This involves executing the system with specific test cases derived from the system's specification and based on the real data that the system is expected to process.
- **Testing as a V&V Activity:** Testing is the most commonly employed activity within the Verification and Validation process.

7.2. Stages of Testing Software testing is typically conducted in several distinct stages:

- **Development or Component Testing:** In this stage, individual software components are tested independently. These components can be functions, objects, or coherent groupings of such entities.
- **System Testing:** This involves testing the system as a complete, integrated whole. Testing for emergent properties (behaviors that arise from the interaction of components but are not present in individual components) is particularly important at this stage.
- **Acceptance Testing:** This final stage of testing involves executing the system using actual customer data. Its purpose is to verify that the system effectively meets the customer's real-world needs and business objectives.

8. Coping with Change

8.1. Inevitability of Change Change is an unavoidable aspect of all large software projects. Several factors contribute to this inevitability:

- **Business Changes:** Evolving business environments frequently lead to new or altered system requirements.
- **New Technologies:** The emergence of new technologies opens up fresh possibilities for improving existing software implementations.
- **Changing Platforms:** Modifications to underlying hardware or software platforms necessitate changes to the application itself.

8.2. Costs of Change Change inherently leads to rework. Therefore, the costs associated with change encompass both the effort involved in rework (e.g., re-analyzing requirements) and the costs of implementing the new functionality.

8.3. Reducing the Costs of Rework Strategies to mitigate the costs of rework include:

- **Change Avoidance:** This involves incorporating activities into the software process that can anticipate potential changes before significant rework becomes necessary. For instance, a prototype system might be developed to demonstrate key features to customers, allowing early feedback and requirement refinement.

- **Change Tolerance:** This strategy designs the software process in a way that allows changes to be accommodated at a relatively low cost. This typically involves some form of incremental development. Proposed changes can be implemented within increments that have not yet been developed. If a change impacts an already developed increment, only that specific (small) increment may need alteration.

9. Software Prototyping

9.1. Definition of a Prototype A prototype is an initial version of a system that is primarily used to demonstrate concepts, explore design options, and elicit feedback.

9.2. Uses of Prototypes Prototypes can be effectively used in various stages of the software development lifecycle:

- **Requirements Engineering:** They assist in eliciting and validating requirements by providing users with a tangible representation of the system.
- **Design Processes:** Prototypes help explore different design alternatives and aid in the development of user interface (UI) designs.
- **Testing Process:** They can be used to run back-to-back tests, comparing the prototype's behavior with that of the full system.

9.3. Benefits of Prototyping The use of prototypes offers several benefits:

- Improved system usability.
- A closer alignment between the system and users' real needs.
- Enhanced design quality.
- Improved maintainability of the final system.
- Reduced overall development effort by identifying issues early.

9.4. Prototype Development Process Prototype development often utilizes rapid prototyping languages or tools. During development, it may involve intentionally omitting certain functionalities:

- **Focus on Uncertainty:** Prototypes should concentrate on areas of the product that are not well-understood.
- **Limited Error Handling:** Error checking and recovery mechanisms may not be fully included in the prototype to expedite development.
- **Functional Focus:** The primary focus is typically on functional requirements, rather than non-functional requirements like reliability or security.

9.5. Throw-away Prototypes It is often recommended that prototypes be discarded after their development and purpose have been served, as they are generally not suitable as a basis for a production system. Reasons for this include:

- **Non-functional Requirements:** It may be impossible to fine-tune the prototype to meet stringent non-functional requirements of a production system.
- **Lack of Documentation:** Prototypes are typically undocumented, making them difficult to maintain or evolve.
- **Degraded Structure:** The prototype's structure often degrades due to rapid, iterative changes during its development.
- **Quality Standards:** Prototypes usually do not meet the normal organizational quality standards required for a production system.

10. Incremental Delivery

10.1. Concept of Incremental Delivery Instead of delivering the entire system as a single, large release, incremental delivery breaks down the development and delivery into smaller increments. Each increment delivers a functional part of the overall required system functionality. User requirements are prioritized, and the highest priority requirements are included in earlier increments. Once the development of a specific increment begins, its requirements are typically frozen, though requirements for subsequent increments can continue to evolve.

10.2. Distinction: Incremental Development vs. Incremental Delivery

- **Incremental Development:** This involves developing the system in increments and evaluating each increment before proceeding to the next. This is a common approach used in agile methods, with evaluation often performed by a user or customer proxy.
- **Incremental Delivery:** This entails deploying an increment for actual use by end-users. This provides a more realistic evaluation of the software's practical utility. However, it can be challenging for replacement systems, as early increments may have less functionality than the system being replaced.

10.3. Advantages of Incremental Delivery

- **Early Customer Value:** Customer value can be delivered with each increment, making system functionality available to users earlier.
- **Elicitation Aid:** Early increments serve as a prototype, helping to elicit and refine requirements for later increments.

- **Reduced Project Risk:** The overall risk of project failure is lowered because issues are identified and addressed incrementally.
- **Focused Testing:** The highest priority system services tend to receive the most rigorous testing, as they are included in early, frequently used increments.

10.4. Problems with Incremental Delivery

- **Common Facilities Identification:** Most systems require a core set of basic facilities utilized across different parts of the system. Since requirements are not detailed until an increment is about to be implemented, it can be difficult to identify these common facilities that all increments will need.
- **Procurement Model Conflict:** The iterative nature of incremental processes, where specification evolves alongside software development, often conflicts with the procurement models used by many organizations. These models frequently require a complete system specification as part of the system development contract before work commences.

11. Process Improvement

11.1. Objectives of Process Improvement The objectives of process improvement initiatives include:

- Understanding, modeling, and enhancing the software process itself.
- Explaining the fundamental principles of software process improvement.
- Explaining how various software process factors influence software quality and productivity.
- Introducing the SEI Capability Maturity Model (CMM) and discussing its influence and applicability.
- Explaining why CMM-based improvement may not be universally applicable.

11.2. Topics Covered in Process Improvement Key topics within process improvement involve:

- Process and product quality.
- Process analysis and modeling.
- Process measurement.
- The SEI process maturity model.
- Process classification.

11.3. General Aspects of Process Improvement Process improvement focuses on:

- Understanding existing processes.
- Introducing changes to processes to achieve organizational objectives, typically centered on quality improvement, cost reduction, and schedule acceleration.
- Historically, most process improvement efforts have concentrated on defect reduction, reflecting an increasing industry focus on quality.
- However, other process attributes can also be the target of improvement.

11.4. Process Improvement Stages A typical process improvement process involves several stages:

- **Process Analysis:** This stage involves modeling and analyzing existing processes, ideally quantitatively.
- **Improvement Identification:** Identifying bottlenecks related to quality, cost, or schedule.
- **Process Change Introduction:** Modifying the process to eliminate the identified bottlenecks.
- **Process Change Training:** Providing training to staff involved in the newly proposed process changes.
- **Change Tuning:** Continuously evolving and refining the implemented process improvements.

12. Process and Product Quality

12.1. Relationship between Process and Product Quality Process quality and product quality are closely related. A good, well-defined process is generally a prerequisite for producing a high-quality product.

- **Manufactured Goods:** For manufactured goods, the process is often the primary determinant of quality.
- **Design-based Activity:** For design-based activities, such as software development, other factors, particularly the capabilities of the designers, also play a significant role.

12.2. Principal Product Quality Factors Several factors influence product quality:

- **Large Projects:** For large projects staffed with teams of "average" capabilities, the development process itself largely determines the product's quality.
- **Small Projects:** For smaller projects, the individual capabilities and skills of the developers are the main determinants of product quality. The development technology used is particularly significant for small projects.

- **Schedule Impact:** In all cases, if an unrealistic or excessively tight schedule is imposed, product quality will inevitably suffer.

13. Process Analysis and Modelling

13.1. Definitions

- **Process Analysis:** The systematic study of existing processes to understand the relationships between different parts of the process and to compare them with other established processes.
- **Process Modelling:** The documentation of a process, which formally records the tasks involved, the roles of individuals, and the entities or artifacts used. Process models can be presented from various perspectives (e.g., activities, deliverables).

13.2. Steps in Process Analysis and Modelling

- **Study Existing Process:** Thoroughly examine an existing process to understand its activities.
- **Produce Abstract Model:** Create an abstract model of the process, typically represented graphically. Multiple views (e.g., activities, deliverables) may be necessary.
- **Analyze Model for Problems:** Analyze the model to identify process problems. This often involves discussing activities with stakeholders.

13.3. Process Analysis Techniques Techniques used for process analysis include:

- **Published Process Models and Standards:** It is often best to begin process analysis with an existing model or standard, which can then be extended or modified.
- **Questionnaires and Interviews:** These methods can gather information, but must be carefully designed, as participants might provide answers they believe are desired.
- **Ethnographic Analysis:** This technique involves gaining process knowledge through direct observation of the process in action.

13.4. Elements of a Process Model A process model can describe activities within a specific process, such as module testing.

14. Process Exceptions

14.1. Nature of Process Exceptions Software processes are complex, and even comprehensive process models cannot effectively represent how to handle all

possible exceptions. Exceptions are unforeseen circumstances that disrupt the normal flow of a process. Examples include:

- Several key personnel falling ill just before a critical review.
- A complete failure of a communication processor, leading to an inability to send/receive email for an extended period.
- An unexpected organizational reorganization.
- A sudden need to respond to an unanticipated request for new proposals.

14.2. Handling Exceptions Under such exceptional circumstances, the defined process model is typically suspended, and managers must use their initiative and judgment to address the exception.

15. Process Measurement

15.1. Importance of Quantitative Data Wherever possible, quantitative data related to the software process should be collected. However, this can be very difficult in organizations without clearly defined process standards, as it's unclear what to measure. A process may need to be defined before any meaningful measurement can occur.

15.2. Purpose of Measurements Process measurements should be used to assess the effectiveness of process improvements. It is crucial to note that measurements should not be the sole driver for improvements; rather, organizational objectives should be the primary driving force behind improvement initiatives.

15.3. Classes of Process Measurement Process measurements can be broadly categorized into three classes:

- **Time Metrics:** These measure the time taken for process activities to be completed, such as calendar time or effort required to complete an activity or the entire process.
- **Resource Utilization Metrics:** These measure the resources required for processes or activities, such as total effort in person-days.
- **Event Metrics:** These measure the number of occurrences of a particular event, such as the number of defects discovered.

15.4. Goal-Question-Metric (GQM) Paradigm The GQM paradigm provides a structured approach for defining and collecting measurements:

- **Goals:** What the organization is striving to achieve. The objective of process improvement is to satisfy these goals.

- **Questions:** Specific areas of uncertainty related to the goals. Process knowledge is essential to derive these questions.
- **Metrics:** The actual measurements to be collected in order to answer the defined questions.

16. The Software Engineering Institute (SEI)

16.1. Background and Mission The Software Engineering Institute (SEI) is a US Defense Department-funded institute affiliated with Carnegie Mellon University. Its primary mission is to promote software technology transfer, particularly to defense contractors.

16.2. Influence of the Maturity Model The SEI proposed its Maturity Model in the mid-1980s, refining it in the early 1990s. This model has been highly influential in the field of software process improvement.

17. The SEI Process Maturity Model

17.1. Maturity Model Levels The SEI Capability Maturity Model (CMM) defines five levels of organizational software process maturity:

- **Level 1: Initial:** At this level, the process is essentially uncontrolled, characterized by an ad-hoc approach.
- **Level 2: Repeatable:** Product management procedures are defined and consistently used, allowing for repeatable successes on similar projects.
- **Level 3: Defined:** Process management procedures and strategies are formally defined and consistently used across the organization.
- **Level 4: Managed:** Quality management strategies are defined and quantitatively used to control the process and product.
- **Level 5: Optimising:** Organizations at this highest level continuously define and use process improvement strategies to achieve ongoing enhancements.

17.2. SEI Model Problems Despite its influence, the SEI model has been noted for several limitations:

- **Focus:** It primarily focuses on project management aspects rather than the specifics of product development.
- **Technology Ignorance:** It tends to overlook the use of modern technologies such as rapid prototyping.
- **Missing Risk Analysis:** It does not explicitly incorporate risk analysis as a key process area.

- **Undefined Applicability Domain:** The model does not clearly define its domain of applicability, leading to questions about its suitability for all types of projects or organizations.

17.3. CMM and ISO 9000 There is a clear correlation between the key process areas identified in the CMM and the quality management processes outlined in ISO 9000. The CMM is more detailed and prescriptive, providing a comprehensive framework for process improvement. Organizations rated at Level 2 in the CMM are generally likely to be compliant with ISO 9000 standards.

17.4. Capability Assessment A significant role of the SEI is to use the CMM to assess the capabilities of contractors bidding for US government defense contracts. The model aims to represent an organization's overall capability, rather than merely the practices used in particular projects. It is important to note that within the same organization, there can often be wide variations in the processes employed. Capability assessment is typically questionnaire-based.

18. Process Classification

Processes can be classified based on their level of formality and support:

- **Informal:** Characterized by no detailed process model, where the development team chooses its own way of working.
- **Managed:** Features a defined process model that actively drives the development process.
- **Methodical:** Processes that are supported by specific development methods, such as HOOD (Hierarchical Object-Oriented Design).
- **Supported:** Processes that are further supported by automated Computer-Aided Software Engineering (CASE) tools.

19. Process Applicability and Choice

19.1. Dependence on Product Type The choice of process should ideally depend on the specific type of product being developed.

- **Large Systems:** For large systems, management challenges are often the principal problem, necessitating a strictly managed process.
- **Smaller Systems:** For smaller systems, a more informal approach may be feasible and appropriate.

19.2. No Universal Process There is no single, universally applicable process that should be standardized across an entire organization. Forcing an

inappropriate process onto a development team can lead to high costs and inefficiencies.

20. Key Points Summary

- Process improvement involves a continuous cycle of process analysis, standardization, measurement, and change.
- Process models serve to describe tasks, activities, roles, exceptions, communication patterns, deliverables, and relationships with other processes.
- Measurements should be purposefully collected to address specific questions about the software process being utilized.
- The three main types of process metrics that can be gathered are time metrics, resource utilization metrics, and event metrics.
- The Software Engineering Institute (SEI) model categorizes software processes into five maturity levels: initial, repeatable, defined, managed, and optimising. It identifies key processes relevant at each level.
- The SEI model is particularly suitable for large systems developed by large teams of engineers and may require modification for other contexts.
- Processes can also be classified as informal, managed, methodical, and improving. This classification can guide the selection of appropriate process tool support.

Software Engineering Notes: Requirements Engineering

UNIT-I Overview

This unit provides an introduction to professional software development, software engineering ethics, and relevant case studies. It also covers software processes, including models, process activities, coping with change, and process improvement. A significant portion is dedicated to Requirements Engineering and System Modelling, which encompasses software requirements (functional and non-functional), requirements elicitation, specification, validation, and change management.

Chapter 4 – Requirements Engineering

This chapter details the following key topics in requirements engineering:

- Functional and non-functional requirements.
- Requirements engineering processes.
- Requirements elicitation.
- Requirements specification.
- Requirements validation.

- Requirements change.

1. Requirements Engineering: Definition and Purpose

Requirements engineering is defined as the systematic process of establishing the services that a customer requires from a system, along with identifying the constraints under which the system operates and is developed. The requirements themselves are the descriptions of the system services and constraints that are produced throughout this engineering process.

2. What is a Requirement?

A requirement can vary significantly in its level of detail and abstraction. It may range from a high-level, abstract statement outlining a service or a system constraint, to a highly detailed mathematical functional specification. This dual nature of requirements is inevitable because they can serve multiple functions:

- **Basis for a Contract Bid:** Requirements might be formulated in a way that allows for interpretation, serving as the foundation for a company's bid for a software development contract. This allows multiple contractors to propose different solutions to meet the client's needs.
- **Basis for the Contract Itself:** Once a contract is awarded, the requirements must be defined in detail to form the binding agreement. The contractor then develops a more detailed system definition for the client to understand and validate the software's functionality.
- Both of these distinct statements, despite their varying levels of detail, can be referred to as "requirements".

3. Types of Requirements

Software requirements are generally categorized into two main types:

- **User Requirements:**
 - These are statements primarily written for customers.
 - They are typically expressed in natural language, supplemented by diagrams, describing the services the system will provide and its operational constraints.
- **System Requirements:**
 - These constitute a structured document that provides detailed descriptions of the system's functions, services, and operational constraints.
 - System requirements define precisely what should be implemented and can therefore form part of a formal contract between the client and the contractor.

4. Functional and Non-functional Requirements

Requirements are further classified based on their nature:

- **Functional Requirements:**
 - These are statements detailing the services the system should provide.
 - They describe how the system should react to specific inputs and its behavior in particular situations.
 - Functional requirements can also explicitly state what the system should *not* do.
 - They describe specific functionality or system services.
 - The nature of functional requirements depends on the type of software, its intended users, and the system environment in which it will operate.
 - Functional *user* requirements might be high-level statements about the system's overall functions.
 - Functional *system* requirements, conversely, should describe the system services in precise detail.
 - **Examples for the MHC-PMS (Mental Health Care - Patient Management System):**
 - A user must be able to search the appointments lists for all clinics.
 - The system shall generate daily, for each clinic, a list of patients expected to attend appointments that day.
 - Each staff member using the system shall be uniquely identified by their 8-digit employee number.
 - **Requirements Imprecision:**
 - Problems arise when requirements are not precisely stated, leading to ambiguity.
 - Ambiguous requirements can be interpreted differently by developers and users.
 - **Example:** The term "search" in the requirement "A user shall be able to search the appointments lists for all clinics".
 - User intention: To search for a patient name across all appointments in all clinics.
 - Developer interpretation: To search for a patient name within an individual clinic, requiring the user to first select the clinic.
 - **Requirements Completeness and Consistency:**
 - Ideally, requirements should be both complete and consistent.

- **Complete:** Requirements should include descriptions of all necessary facilities.
- **Consistent:** There should be no conflicts or contradictions within the descriptions of the system's facilities.
- In practical scenarios, achieving a truly complete and consistent requirements document is often impossible.
- **Non-functional Requirements:**
 - These define system properties and constraints, such as reliability, response time, and storage requirements.
 - Constraints can also relate to I/O device capability, system representations, and other factors.
 - Process requirements may also be specified, mandating specific development tools (e.g., IDE), programming languages, or development methodologies.
 - Non-functional requirements can often be more critical than functional ones; if they are not met, the system may become useless.
 - They typically apply to the system as a whole rather than just individual features or services.
 - They often relate to the emergent properties of the system.
 - **Implementation Impact:**
 - Non-functional requirements can significantly influence the overall architecture of a system, rather than just individual components. For instance, meeting performance requirements might necessitate organizing the system to minimize communication between components.
 - A single non-functional requirement, like a security requirement, might lead to the generation of multiple related functional requirements that define necessary system services. It can also impose restrictions on existing requirements.
 - **Non-functional Classifications:**
 - **Product Requirements:** These specify how the delivered product must behave, covering aspects such as execution speed, reliability, and security.
 - **Organizational Requirements:** These arise from organizational policies and procedures, including process standards that must be used or specific implementation requirements.
 - **External Requirements:** These originate from factors external to the system and its development process, such as interoperability requirements, legislative requirements, or ethical guidelines.

- **Examples of Non-functional Requirements in the MHC-PMS:**
 - **Product Requirement:** "The MHC-PMS shall be available to all clinics during normal working hours (Mon–Fri, 0830–17.30). Downtime within normal working hours shall not exceed five seconds in any one day".
 - **Organizational Requirement:** "Users of the MHC-PMS system shall authenticate themselves using their health authority identity card".
 - **External Requirement:** "The system shall implement patient privacy provisions as set out in HStan-03-2006-priv".
- **Goals and Verifiable Non-functional Requirements:**
 - Non-functional requirements can be challenging to state precisely, and imprecise requirements are difficult to verify.
 - **Goal:** A general intention of the user, such as "ease of use".
 - **Verifiable Non-functional Requirement:** A precise statement that includes a measure which can be objectively tested.
 - Goals are valuable as they convey the intentions of the system users to developers.
 - **Example (Usability):**
 - Goal: "The system should be easy to use by medical staff and should be organized in such a way that user errors are minimized".
 - Testable Non-functional Requirement: "Medical staff shall be able to use all the system functions after four hours of training. After this training, the average number of errors made by experienced users shall not exceed two per hour of system use".
- **Metrics for Specifying Non-functional Requirements:**
 - **Property:** Speed
 - **Measures:** Processed transactions/second, User/event response time, Screen refresh time.
 - **Property:** Size
 - **Measures:** Mbytes, Number of ROM chips.
 - **Property:** Ease of use
 - **Measures:** Training time, Number of help frames.
 - **Property:** Reliability
 - **Measures:** Mean time to failure, Probability of unavailability, Rate of failure occurrence, Availability.
 - **Property:** Robustness
 - **Measures:** Time to restart after failure, Percentage of events causing failure, Probability of data corruption on failure.

- **Property:** Portability
 - **Measures:** Percentage of target dependent statements, Number of target systems.
- **Domain Requirements:**
 - These are constraints on the system that originate from the system's operational domain.
 - For example, a train control system must account for braking characteristics under different weather conditions.
 - Domain requirements can introduce new functional requirements, impose constraints on existing requirements, or define specific computations.
 - Failure to satisfy domain requirements can render the system unworkable.
 - **Problems with Domain Requirements:**
 - **Understandability:** Requirements are often expressed in the specialized language of the application domain, which software engineers developing the system may not understand.
 - **Implicitness:** Domain specialists possess such deep understanding of their area that they may not explicitly state certain domain requirements, assuming them to be obvious.

5. Key Points about Requirements

- Requirements for a software system define what the system should do and establish constraints on its operation and implementation.
- Functional requirements specify the services a system must provide or describe how particular computations are performed.
- Non-functional requirements frequently constrain the system being developed and the development process used. They are often related to the emergent properties of the system and, as such, apply to the system as a whole.

6. The Software Requirements Document

The software requirements document serves as the official statement outlining what is required of the system developers. It should include both a definition of user requirements and a detailed specification of system requirements.

Crucially, it is not a design document; its primary purpose is to define *what* the system should do, rather than *how* it should do it.

- **Users of a Requirements Document:**
 - **System test engineers:** Use the requirements to develop validation tests for the system.

- **Managers:** Use the requirements document to plan bids for the system and to plan the system development process.
- **System engineers:** Use the requirements to understand what system is to be developed.
- **System customers:** Specify the requirements and review them to ensure they meet their needs, also specifying changes to the requirements.
- **System maintenance engineers:** Use the requirements to help understand the system and the relationships between its parts.
- **Requirements Document Variability:**
 - The information included in a requirements document can vary based on the type of system being developed and the chosen development approach.
 - Systems developed incrementally typically feature less detail in their initial requirements documents.
 - Standard formats for requirements documents, such as those designed by IEEE, exist but are primarily applicable to requirements for large-scale systems engineering projects.
- **Structure of a Requirements Document:**
 - **Preface:** Defines the expected readership, describes the document's version history, including the rationale for new versions, and summarizes changes.
 - **Introduction:** Describes the need for the system, briefly outlines its functions, explains its interaction with other systems, and describes how it aligns with the organization's business or strategic objectives.
 - **Glossary:** Defines technical terms used in the document, assuming no prior experience or expertise of the reader.
 - **User Requirements Definition:** Describes the services provided to the user, including non-functional system requirements. This section may use natural language, diagrams, or other customer-understandable notations. Product and process standards that must be adhered to should also be specified.
 - **System Architecture:** Presents a high-level overview of the anticipated system architecture, showing the distribution of functions across system modules and highlighting reused architectural components.
 - **System Requirements Specification:** Describes the functional and non-functional requirements in more detail. Further detail may be added to non-functional requirements if necessary, and interfaces to other systems may be defined.
 - **System Models:** May include graphical system models illustrating relationships between system components and between the system

and its environment. Examples include object models, data-flow models, or semantic data models.

- **System Evolution:** Describes the fundamental assumptions underlying the system and any anticipated changes due to hardware evolution, changing user needs, etc. This section assists system designers in avoiding design decisions that might constrain future changes.
- **Appendices:** Provide detailed, specific information related to the application, such as hardware and database descriptions. Hardware requirements define minimal and optimal system configurations, while database requirements define the logical organization of data and relationships between data.
- **Index:** May include multiple indexes, such as an alphabetic index, an index of diagrams, and an index of functions.

7. Requirements Specification

Requirements specification is the process of formally documenting user and system requirements in a requirements document.

- User requirements must be understandable by end-users and customers who may not have a technical background.
- System requirements are more detailed and may contain more technical information.
- Since requirements can become part of a contract for system development, it is crucial that they are as complete as possible.
- **Ways of Writing a System Requirements Specification:**
 - **Natural Language:** Requirements are written as numbered sentences in natural language, with each sentence expressing a single requirement.
 - **Structured Natural Language:** Requirements are written in natural language using a standard form or template, where each field provides specific information about an aspect of the requirement.
 - **Design Description Languages:** This approach employs a language similar to a programming language but with more abstract features to specify requirements by defining an operational model of the system. This method is now rarely used, though it can be beneficial for interface specifications.
 - **Graphical Notations:** Graphical models, complemented by text annotations, are used to define functional requirements. UML use case and sequence diagrams are common examples.

- **Mathematical Specifications:** These notations are based on mathematical concepts like finite-state machines or sets. While these unambiguous specifications can reduce ambiguity in a requirements document, most customers typically do not understand formal specifications, making them reluctant to accept them as a system contract or verify their alignment with their needs.

8. Requirements and Design

In principle, requirements should specify *what* the system should do, while design should describe *how* it accomplishes this. However, in practice, requirements and design are often inseparable:

- A system architecture might be specifically designed to structure the requirements.
- The system may need to inter-operate with other systems, which can generate design requirements.
- The selection of a particular architecture to fulfill non-functional requirements may itself be a domain requirement, potentially due to regulatory mandates.

9. Natural Language Specification

Requirements are commonly written as natural language sentences, supplemented by diagrams and tables. This approach is widely used because natural language is expressive, intuitive, and universally understood, making requirements accessible to users and customers.

- **Guidelines for Writing Requirements:**
 - Invent and consistently use a standard format for all requirements.
 - Use language consistently; for example, use "shall" for mandatory requirements and "should" for desirable ones.
 - Use text highlighting to emphasize key parts of the requirement.
 - Avoid using computer jargon.
 - Include an explanation (rationale) for why a requirement is necessary.
- **Problems with Natural Language:**
 - **Lack of Clarity:** Achieving precision in natural language can make the document difficult to read.
 - **Requirements Confusion:** Functional and non-functional requirements often become mixed together.
 - **Requirements Amalgamation:** Several distinct requirements may be combined and expressed together.

- **Example Requirements for an Insulin Pump Software System:**
 - "3.2 The system shall measure the blood sugar and deliver insulin, if required, every 10 minutes. (Changes in blood sugar are relatively slow so more frequent measurement is unnecessary; less frequent measurement could lead to unnecessarily high sugar levels.)".
 - "3.6 The system shall run a self-test routine every minute with the conditions to be tested and the associated actions defined in Table 1. (A self-test routine can discover hardware and software problems and alert the user to the fact the normal operation may be impossible.)".

10. Structured Specifications

Structured specifications are an approach to writing requirements where the writer's freedom is limited, and requirements are written in a standardized manner. This method is effective for certain types of requirements, such as those for embedded control systems, but can sometimes be too rigid for business system requirements.

- **Form-based Specifications:**
 - These typically include:
 - Definition of the function or entity.
 - Description of inputs and their sources.
 - Description of outputs and their destinations.
 - Information about the data needed for the computation and other entities used.
 - Description of the action to be taken.
 - Pre-conditions and post-conditions (if applicable).
 - Side effects (if any) of the function.
- **Tabular Specification:**
 - Used to supplement natural language specifications.
 - Particularly useful for defining multiple possible alternative courses of action.
 - **Example:** An insulin pump system that bases its computations on the rate of change of blood sugar level can use a tabular specification to explain how to calculate insulin requirements for various scenarios.

11. Requirements Engineering Processes

The processes employed for Requirements Engineering (RE) vary significantly based on the application domain, the individuals involved, and the organization

developing the requirements. However, several generic activities are common to most RE processes:

- Requirements elicitation.
- Requirements analysis.
- Requirements validation.
- Requirements management.

In practice, RE is an iterative activity where these processes are interleaved. A spiral view can represent the iterative nature of the requirements engineering process.

12. Requirements Elicitation and Analysis

Also known as requirements elicitation or requirements discovery. This phase involves technical staff collaborating with customers to gather information about the application domain, the services the system should provide, and its operational constraints. This often includes interactions with a wide range of individuals referred to as "stakeholders," such as end-users, managers, maintenance engineers, domain experts, and trade unions.

- **Problems of Requirements Analysis / Elicitation:**
 - Stakeholders may not precisely know what they truly want.
 - Stakeholders often express requirements using their own specific terminology.
 - Different stakeholders may have conflicting requirements.
 - Organizational and political factors can influence system requirements.
 - Requirements tend to change throughout the analysis process, with new stakeholders emerging and the business environment evolving.
- **Process Activities (Stages):**
 - **Requirements Discovery:** Involves interacting with stakeholders to uncover their requirements, including domain-specific requirements. This is the process of gathering information about required and existing systems and distilling user and system requirements from it. Interaction occurs with system stakeholders ranging from managers to external regulators. Systems typically involve a range of stakeholders.
 - **Requirements Classification and Organization:** Groups related requirements and arranges them into coherent clusters.
 - **Prioritization and Negotiation:** Involves prioritizing requirements and resolving any conflicts that arise.

- **Requirements Specification:** The process of documenting the requirements, which then serves as input for the next iteration of the spiral process.
- **Stakeholders in the MHC-PMS Example:**
 - Patients whose information is recorded in the system.
 - Doctors responsible for assessing and treating patients.
 - Nurses who coordinate consultations with doctors and administer treatments.
 - Medical receptionists who manage patient appointments.
 - IT staff responsible for system installation and maintenance.
 - A medical ethics manager who ensures system compliance with ethical patient care guidelines.
 - Health care managers who obtain management information from the system.
 - Medical records staff responsible for system information maintenance, preservation, and proper implementation of record-keeping procedures.
- **Interviewing:**
 - Formal or informal interviews with stakeholders are a common part of most RE processes.
 - **Types of Interviews:**
 - **Closed Interviews:** Based on a predetermined list of questions.
 - **Open Interviews:** Allow various issues to be explored with stakeholders.
 - **Effective Interviewing:**
 - Be open-minded and avoid preconceived ideas about requirements.
 - Be willing to listen carefully to stakeholders.
 - Prompt interviewees to initiate discussions using "springboard questions," requirements proposals, or by collaboratively working on a prototype system.
 - **Interviews in Practice:**
 - Typically, a mix of closed and open-ended interviewing techniques is used.
 - Interviews are effective for gaining an overall understanding of what stakeholders do and how they might interact with the system.
 - They are generally *not* effective for understanding detailed domain requirements because:
 - Requirements engineers may not understand specific domain terminology.

- Some domain knowledge is so fundamental to domain specialists that they find it difficult to articulate or do not deem it necessary to articulate.
- **Scenarios:**
 - Scenarios are real-life examples illustrating how a system can be used.
 - **Components of a Scenario:**
 - A description of the starting situation.
 - A description of the normal flow of events.
 - A description of what could go wrong.
 - Information about other concurrent activities.
 - A description of the system's state upon completion of the scenario.
 - **Example: Scenario for Collecting Medical History in MHC-PMS:**
 - **Initial Assumption:** A patient has a record created by a medical receptionist, who has collected personal information. A nurse is logged in and collecting medical history.
 - **Normal Flow:** The nurse searches for the patient (using family name, then given name and date of birth if needed). The nurse selects the option to add medical history and follows system prompts to enter information on consultations, existing conditions (menu selection), medication (menu selection), allergies (free text), and home life (form).
 - **What Can Go Wrong:**
 - Patient record not found: Nurse creates a new record and enters personal information.
 - Conditions/medication not in menu: Nurse selects 'other' and enters free text.
 - Patient cannot/will not provide information: Nurse records this in free text. System prints standard exclusion form stating treatment limitations, which is signed and given to patient.
 - **Other Activities:** Record may be consulted (but not edited) by other staff while information is being entered.
 - **System State on Completion:** User logged on. Patient record with medical history entered into database. A record of the session (start/end time, nurse involved) added to the system log.
- **Use Cases:**

- Use cases are a scenario-based technique within the Unified Modeling Language (UML).
- They identify the actors involved in an interaction and describe the interaction itself.
- A complete set of use cases should describe all possible interactions with the system.
- Use cases are typically represented as high-level graphical models supplemented by more detailed tabular descriptions.
- Sequence diagrams can be used to add further detail to use cases by illustrating the sequence of event processing within the system.

13. Requirements Validation

Requirements validation is the process concerned with demonstrating that the defined requirements accurately represent the system that the customer truly desires. This step is crucial because the costs associated with fixing requirements errors are significantly high; fixing a requirements error after system delivery can cost up to 100 times more than fixing an implementation error.

- **Requirements Checking Criteria:**
 - **Validity:** Does the system provide the functions that best support the customer's needs?.
 - **Consistency:** Are there any conflicts or contradictions among the requirements?.
 - **Completeness:** Are all functions required by the customer included in the specification?.
 - **Realism:** Can the requirements be implemented given the available budget and technology?.
 - **Verifiability:** Can the requirements be objectively checked or tested?.
- **Requirements Validation Techniques:**
 - **Requirements Reviews:** Involve systematic manual analysis of the requirements.
 - **Prototyping:** Uses an executable model of the system to check requirements.
 - **Test-Case Generation:** Involves developing tests for requirements to assess their testability.
- **Requirements Reviews:**
 - Regular reviews should be conducted throughout the requirements definition phase.
 - Both client and contractor staff should participate in these reviews.

- Reviews can be formal (involving completed documents) or informal.
- Effective communication among developers, customers, and users can help resolve problems at an early stage.
- **Review Checks:**
 - **Verifiability:** Is the requirement realistically testable?.
 - **Comprehensibility:** Is the requirement properly understood by all stakeholders?.
 - **Traceability:** Is the origin of the requirement clearly stated?.
 - **Adaptability:** Can the requirement be changed without a significant impact on other requirements?.

14. Requirements Management

Requirements management is the ongoing process of controlling and adapting to changing requirements throughout the requirements engineering process and system development. New requirements frequently emerge during system development and even after the system is deployed. It is essential to track individual requirements, maintain links between dependent requirements to assess the impact of changes, and establish a formal process for proposing changes and linking them to system requirements.

- **Reasons for Changing Requirements:**
 - **Evolving Environment:** The business and technical environment of a system invariably changes after installation. This can include new hardware, interfaces with other systems, shifts in business priorities (leading to changes in required system support), and new legislation or regulations that the system must comply with.
 - **Stakeholder Differences:** The individuals who fund a system (customers) and the individuals who use it (end-users) are rarely the same. Customers impose requirements based on organizational and budgetary constraints, which may conflict with end-user needs. Consequently, new features might need to be added post-delivery to better support users and help the system achieve its goals.
 - **Diverse User Community:** Large systems typically serve a diverse user community, with many users having distinct and potentially conflicting or contradictory requirements and priorities. The final system requirements are usually a compromise. Experience often reveals that the balance of support provided to different users needs adjustment.
 - Requirements evolution is a natural part of the software lifecycle.
- **Requirements Management Planning:**

- This activity establishes the necessary level of detail for requirements management.
- Key decisions made during planning include:
 - **Requirements Identification:** Each requirement must be uniquely identified to enable cross-referencing with other requirements.
 - **Change Management Process:** This defines the set of activities used to assess the impact and cost of proposed changes.
 - **Traceability Policies:** These policies define the relationships that should be recorded between each requirement and between the requirements and the system design.
 - **Tool Support:** Specifies the tools that may be used, ranging from specialized requirements management systems to spreadsheets and simple database systems.
- **Requirements Change Management Process:**
 - This process focuses on deciding whether a proposed requirements change should be accepted.
 - **Stages of the process:**
 - **Problem Analysis and Change Specification:** The proposed problem or change is analyzed to validate its legitimacy. This analysis is communicated back to the change requestor, who may then provide a more specific proposal or withdraw the request.
 - **Change Analysis and Costing:** The potential effects of the proposed change are assessed using traceability information and general knowledge of the system requirements. Following this analysis, a decision is made regarding whether to proceed with the requirements change.
 - **Change Implementation:** The requirements document, and where necessary, the system design and implementation, are modified. Ideally, the document should be organized to facilitate easy implementation of changes.

15. Key Points about Requirements Engineering Activities

- A variety of techniques can be used for requirements elicitation, including interviews, scenarios, use-cases, and ethnography.
- Requirements validation is the process of checking requirements for their validity, consistency, completeness, realism, and verifiability.
- Changes in business, organizational, and technical environments inevitably lead to changes in software system requirements.

Requirements management is the crucial process of managing and controlling these changes.